

P1147R1

Printing volatile Pointers

Published Proposal, 2021-09-25

Author:

[Bryce Adelstein LeLach](#) ([NVIDIA](#))

Source:

github.com/wg21_p1147_printing_volatile_pointers/blob/master/printing_volatile_pointers.bs

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

LEWG

Table of Contents

1 Introduction

2 Wording

3 Acknowledgements

§ 1. Introduction

Printing pointers to `volatile` types with standard library output streams has unexpected results. Consider the following code:

```
#include <iostream>
```

```
int main()  
{
```

```

        int* p0 = reinterpret_cast<        int*>(0xdeadbeef);
volatile int* p1 = reinterpret_cast<volatile int*>(0xdeadbeef);

std::cout << p0 << std::endl;
std::cout << p1 << std::endl;
}

```

This produces the following output:

```

0xdeadbeef
1

```

What happened here? Well, `basic_ostream` has an `operator<<` for both `const void*` and `bool`:

27.7.5.1 Class template `basic_ostream`

[ostream]

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ostream : virtual public basic_ios<charT, traits> {
    public:
        // ...

        // [ostream.formatted], formatted output
        // ...
        basic_ostream<charT, traits>& operator<<(bool n);
        basic_ostream<charT, traits>& operator<<(const void* p);
        // ...

        // ...
    };
}

```

For `std::cout << p0`, `p0` (an `int*`) is implicitly converted to `const void*` and `operator<<(const void*)` is called. However, for `std::cout << p1`, the `operator<<(const void*)` overload is not a match, as it discards qualifiers. Instead, the best match is `operator<<(bool)`, so `p1` (a `volatile int*`) is implicitly converted to `bool` and `operator<<(bool)` is called.

I suggest we add a new `operator<<(const volatile void*)` overload that `const_casts` away `volatile` and calls the `operator<<(const void*)` overload. Initially, I explored modifying the underlying `<locale>` methods and changing the existing `operator<<(const void*)` to `operator<<(const volatile void*)`, however, early feedback from other committee members

pushed me away from this direction, as there was concern that modifying `<locale>` would be an ABI breaking change.

Note that `const_casting` away `volatile` is safe here. We are not accessing or printing the value of the object that the pointer points to, we are just printing the value of the pointer itself.

§ 2. Wording

Modify [\[ostream.general\]](#) as follows:

```
namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ostream : virtual public basic_ios<charT, traits> {
    public:
        // ...

        // [ostream.formatted], formatted output
        basic_ostream& operator<<(basic_ostream& (*pf)(basic_ostream&));
        basic_ostream& operator<<(basic_ios<charT, traits>& (*pf)(basic_ios<
        basic_ostream& operator<<(ios_base& (*pf)(ios_base&));

        basic_ostream& operator<<(bool n);
        basic_ostream& operator<<(short n);
        basic_ostream& operator<<(unsigned short n);
        basic_ostream& operator<<(int n);
        basic_ostream& operator<<(unsigned int n);
        basic_ostream& operator<<(long n);
        basic_ostream& operator<<(unsigned long n);
        basic_ostream& operator<<(long long n);
        basic_ostream& operator<<(unsigned long long n);
        basic_ostream& operator<<(float f);
        basic_ostream& operator<<(double f);
        basic_ostream& operator<<(long double f);

        basic_ostream& operator<<(const void* p);
        basic_ostream& operator<<(const volatile void* val);
        basic_ostream& operator<<(nullptr_t);
        basic_ostream& operator<<(basic_streambuf<char_type, traits>* sb);

        // ...
    };

    // ...
}
```

Modify [\[ostream.inserters\]](#) as follows:

...

```
basic_ostream& operator<<(nullptr_t);
```

11 *Effects:* Equivalent to:

```
return *this << s;
```

where `s` is an implementation-defined NTCTS.

```
basic_ostream& operator<<(const volatile void* val);
```

12 *Effects:* Equivalent to:

```
return operator<<(const_cast<const void*>(val));
```

§ 3. Acknowledgements

Thanks to JF Bastien, Marshall Clow, Billy O’Neal, Louis Dionne, Jonathan Wakely, and Jeff Garland for reviewing and providing feedback on this paper.